Data Structures

Part 4: Data Classes

Similar to dictionaries, but trading flexibility for consistency and better code completion.

Data Classes

A data class is a class designed primarily to store a related collection of data.

Unlike dictionaries, data classes define exactly which fields an object is expected to have, reducing mistakes caused by misspelled or inconsistent dictionary keys.

Type annotations can also describe what type of data should be stored in each field, making the structure easier to understand and allowing development tools to identify some mistakes.

Data classes are especially useful when your program repeatedly works with the same type of structured data.

Importing `dataclass`

Data classes are provided by Python's built-in **`dataclasses`** module.

The **`dataclass`** decorator can be imported using **`from dataclasses import dataclass`**.

Once imported, **`@dataclass`** can be used when defining a class.

```
from dataclasses import dataclass
```

Creating a Data Class with `@dataclass`

A data class is created by placing the **`@dataclass`** decorator directly above a **`class`** definition.

The decorator tells Python to automatically create common class functionality based on the fields you define.

For example, Python automatically creates the constructor needed to give those fields their initial values.

```
from dataclasses import dataclass


@dataclass
class Student:
    name: str
    grade: int
```

Defining Fields

Fields are the individual pieces of data that each instance of a data class stores.

Each field is defined with a name followed by a type annotation, such as `name: str` or `grade: int`.

Type annotations clearly communicate what type of value is expected for each field and can help code editors and type-checking tools identify mistakes.

```
from dataclasses import dataclass


@dataclass
class Student:
    name: str
    grade: int
    average: float
```

Dataclasses Are Templates

A data class functions as a template that describes what information should be stored for a particular type of data.

Every instance created from the data class follows the same structure while storing its own values.

This makes data classes useful when a program needs to keep track of many records containing the same kinds of information.

```
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    grade: int

student1 = Student("Alice", 11)
student2 = Student("Bob", 12)
```

Creating Instances

Once a data class has been defined, you can create instances of it just like a regular class.

Values are provided for the fields when the instance is created, usually in the same order that the fields were defined.

Each instance is a separate object containing its own data.

```
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    grade: int

student = Student("Alice", 11)

print(student)
```

OUTPUT

```
Student(name='Alice', grade=11)
```

Accessing Fields with Dot Notation

The fields stored in a data class instance can be accessed using dot notation.

Write the instance variable followed by a period . and the name of the field you want to access.

This provides a clear way to retrieve individual pieces of data from an instance.

```
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    grade: int

student = Student("Alice", 11)

print(student.name)
print(student.grade)
```

OUTPUT

```
Alice
11
```

Changing Fields with Dot Notation

By default, the fields of a data class can be changed after an instance has been created.

Use dot notation to access the field and the assignment operator = to give it a new value.

This changes the field for that specific instance without affecting other instances of the data class.

```
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    grade: int

student = Student("Alice", 11)

student.grade = 12

print(student)
```

OUTPUT

```
Student(name='Alice', grade=12)
```